

SOFTWARE REQUIREMENTS SPECIFICATION

POS, KDS & Branch Engine

Restaurant & Coffee Shop Ordering Platform

Document version: 1.0

Status: Phase 1 & 2 implemented and verified; Phases 3–4 specified, not yet built

Prepared from: POS/KDS Master Implementation Plan and Architecture Blueprint (source documents), plus the working reference implementation

Table of Contents

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Scope.....	3
1.3 Definitions, Acronyms, and Abbreviations.....	3
1.4 References.....	4
1.5 Overview.....	4
2. Overall Description.....	5
2.1 Product Perspective.....	5
2.2 Product Functions.....	5
2.3 User Classes and Characteristics.....	5
2.4 Operating Environment.....	5
2.5 Design and Implementation Constraints.....	6
2.6 Assumptions and Dependencies.....	6
3. Functional Requirements.....	7
3.1 Tenancy, Branch & Menu Management.....	7
3.2 POS Ordering (Cashier).....	7
3.3 Kitchen Display System (KDS).....	8
3.4 Real-Time Sync Engine.....	8
3.5 Hardware Integration.....	8
3.6 Inventory & Add-ons.....	9
4. External Interface Requirements.....	10
4.1 User Interfaces.....	10
4.2 Hardware Interfaces.....	10
4.3 Software Interfaces.....	10
4.4 Communication Interfaces.....	10
5. Non-Functional Requirements.....	11
5.1 Performance.....	11
5.2 Reliability & Availability.....	11
5.3 Security.....	11
5.4 Scalability.....	11
5.5 Usability.....	11
5.6 Maintainability.....	11
6. System Architecture.....	12
6.1 Topology.....	12
6.2 Topology Decision & Rationale.....	12
6.3 Technology Stack.....	12
6.4 Known Gaps Against This Specification.....	12
7. Data Requirements.....	14
Appendix A — REST API Reference (Phase 1-2, Implemented).....	17
A.1 Order Status Lifecycle.....	17

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for a cashier / counter ordering system for restaurants and coffee shops, covering the Point of Sale (POS) cashier interface, the Kitchen Display System (KDS), and the branch-local engine that synchronizes them in real time. It consolidates two prior planning documents — the POS/KDS Master Implementation Plan and the POS & KDS System Architecture Blueprint — into a single reconciled specification, and reflects the architecture actually implemented and tested in the accompanying reference codebase.

1.2 Scope

The system enables a cashier to take dine-in or takeaway orders on a touch-friendly interface, transmit them instantly to a kitchen ticket display, and track each item through preparation to service. It is designed as a multi-tenant SaaS product: a single deployment serves many restaurant/coffee-shop brands (tenants), each with one or more physical branches. The specification below is organized by implementation phase, matching the roadmap in the source Master Plan:

- Phase 1 — Core multi-tenant backend, menu management, table management, and POS ordering.
- Phase 2 — Kitchen Display System and the real-time branch sync engine.
- Phase 3 — Hardware integration: receipt/kitchen printers, cash drawer, card payment terminals.
- Phase 4 — Ingredient-level inventory, low-stock alerts, end-of-day reconciliation, QR self-ordering.

Phases 1 and 2 are implemented and verified in the reference codebase referred to throughout this document. Phases 3 and 4 are specified here but not yet built; their requirements are included so implementation can proceed without re-deriving architecture decisions.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
POS	Point of Sale — the cashier-facing ordering application.
KDS	Kitchen Display System — the kitchen-facing ticket application.
Edge Server	A lightweight server process running on branch-local hardware (Mini-PC / POS terminal) that hosts the local WebSocket hub, the branch's local database, and hardware bridging.
Tenant	A restaurant or coffee shop brand/company using the platform; may own multiple branches.
Branch	A single physical location belonging to a tenant.
RLS	Row-Level Security — a PostgreSQL feature enforcing tenant/branch data isolation at the database layer.
ESC/POS	A common command protocol for thermal receipt and kitchen printers.
WSS / WS	WebSocket (Secure) — full-duplex protocol used for real-time order and status sync.
Ticket	A kitchen-facing representation of an order (or order line) awaiting preparation.

Table 1.1 — Glossary of terms used throughout this specification.

1.4 References

- POS, KDS & Branch Engine — Master Implementation Plan (source document, client-supplied).
- POS & KDS System Architecture Blueprint (source document, client-supplied).
- Reference implementation: pos-kds-system codebase (backend/ and frontend/), Phases 1–2.
- System Architecture document (companion PDF): topology, order sync flow, and database schema diagrams.

1.5 Overview

Section 2 describes the product at a high level. Section 3 specifies functional requirements, organized by feature area and mapped to roadmap phase. Section 4 specifies external interfaces. Section 5 specifies non-functional requirements. Section 6 describes the system architecture and the rationale for the topology chosen. Section 7 specifies data requirements. Appendix A summarizes the REST API implemented to date.

2. Overall Description

2.1 Product Perspective

The product is a new, self-contained multi-tenant SaaS system; it does not replace or integrate with an existing legacy POS at this stage. It is composed of three cooperating tiers:

- Cloud backend (planned) — cross-branch reporting, billing, tenant administration, and the system of record once branch data is replicated upstream.
- Branch edge server (built) — the local source of truth for a single branch: REST API, WebSocket sync hub, local database, and (from Phase 3) hardware bridging.
- Client applications (built) — browser-based Cashier (POS) and Kitchen (KDS) interfaces running on touch terminals, tablets, or laptops on the branch LAN.

This three-tier split was a deliberate reconciliation of the two source documents, which proposed different sync topologies (see Section 6.2 for the comparison and rationale).

2.2 Product Functions

- Take dine-in and takeaway orders against a branch-specific menu.
- Transmit orders to the kitchen in real time, with no perceptible delay under normal LAN conditions.
- Track each order line through a defined preparation lifecycle.
- Maintain table state (available / occupied / dirty) in step with order activity.
- Continue operating during a WAN (internet) outage, since cashier and kitchen communicate over the local branch network.
- (Phase 3) Print receipts and kitchen tickets, open the cash drawer, and process card payments.
- (Phase 4) Deduct ingredient-level stock as orders are placed and surface low-stock alerts.

2.3 User Classes and Characteristics

User class	Characteristics	Primary needs
Cashier	Front-of-house staff; minimal training assumed	Fast order entry, table selection, sending tickets to kitchen
Kitchen staff	Back-of-house staff; works under time pressure	At-a-glance ticket status, one-tap status advancement
Branch manager	Oversees a single branch	Menu and table configuration, end-of-day reconciliation (Phase 4)
Tenant administrator	Oversees multiple branches for a brand	Cross-branch reporting, billing (cloud tier, planned)

Table 2.1 — User classes.

2.4 Operating Environment

- Client applications: any modern Chromium- or WebKit-based browser (desktop, tablet, or touch POS terminal); Progressive Web App with offline asset caching.

- Branch edge server: Node.js runtime on a branch-local Mini-PC or POS terminal; must remain reachable on the branch LAN by all client devices.
- Cloud backend (planned): containerized Node.js/Go services behind a REST/GraphQL API, backed by managed PostgreSQL and Redis.

2.5 Design and Implementation Constraints

- Tenant data isolation must be enforced at the database layer (Row-Level Security in PostgreSQL for the cloud tier), not only in application code.
- Real-time order sync between Cashier and Kitchen must not depend on WAN/internet availability.
- Hardware access (raw TCP printers, USB card readers) requires a component with raw socket access; browsers cannot provide this directly, which is why an edge server process is required rather than a browser-only architecture.
- Client applications must run with zero local installation beyond a browser.

2.6 Assumptions and Dependencies

- Each branch has a stable local area network connecting cashier terminals, kitchen displays, and the edge server.
- Internet connectivity, where present, is used for cloud replication and reporting but is not required for a branch to take and fulfill orders.
- Card payment integration (Phase 3) depends on a third-party terminal SDK (e.g., Stripe Terminal, Adyen) and is out of scope for this document's functional detail beyond the integration points listed in Section 4.2.

3. Functional Requirements

Requirements are grouped by feature area. Each requirement has a unique ID, a priority (MoSCoW: Must / Should / Could), and the roadmap phase it belongs to. Requirements marked Phase 1–2 are implemented and verified in the reference codebase; Phase 3–4 requirements are specified for future implementation.

3.1 Tenancy, Branch & Menu Management

ID	Requirement	Priority	Phase
FR-1.1	The system shall support multiple tenants, each with an isolated set of branches, tables, menu items, and orders.	Must	1
FR-1.2	The system shall support multiple branches per tenant, each with its own currency, timezone, and tax number.	Must	1
FR-1.3	The system shall allow branch staff to view an up-to-date menu scoped to their branch, including name, category, price, and availability.	Must	1
FR-1.4	The system shall allow a menu item to be marked unavailable without deleting it.	Should	1
FR-1.5	The system shall maintain a table floorplan per branch, including table number, zone, and seating capacity.	Must	1
FR-1.6	The system shall track table status as one of: available, occupied, dirty.	Must	1

Table 3.1 — Tenancy, branch, and menu requirements.

3.2 POS Ordering (Cashier)

ID	Requirement	Priority	Phase
FR-2.1	The cashier shall be able to start a new order as either dine-in (with a selected table) or takeaway.	Must	1
FR-2.2	The cashier shall be able to add menu items to an order and adjust line-item quantity before sending.	Must	1
FR-2.3	The system shall calculate the order total from item prices and quantities in real time as the cart changes.	Must	1
FR-2.4	The cashier shall be able to filter the menu by category while building an order.	Should	1
FR-2.5	Upon sending an order, the system shall mark the associated table as occupied.	Must	1
FR-2.6	The cashier shall be able to mark an order as paid, which shall set the associated table to dirty.	Must	1
FR-2.7	The system shall prevent sending an order with zero items, or a dine-in order with no table selected.	Must	1

Table 3.2 — Cashier / POS requirements.

3.3 Kitchen Display System (KDS)

ID	Requirement	Priority	Phase
FR-3.1	The system shall display newly sent orders on the kitchen screen within one second under normal LAN conditions.	Must	2
FR-3.2	Each kitchen ticket shall display the order identifier, order type (dine-in table or takeaway), and all line items with quantity.	Must	2
FR-3.3	Each order line shall progress through the statuses: Placed → In Prep → Ready → Served, advanced by kitchen staff with a single action per step.	Must	2
FR-3.4	The system shall display an elapsed-time indicator per ticket and visually flag tickets exceeding a configurable time threshold.	Should	2
FR-3.5	An order shall automatically close (move off the active board) once every line item reaches Served.	Must	2
FR-3.6	The KDS shall play an audible alert when a new ticket arrives.	Should	2

Table 3.3 — Kitchen Display System requirements.

3.4 Real-Time Sync Engine

ID	Requirement	Priority	Phase
FR-4.1	The system shall broadcast order creation and order-item status changes to all connected clients on the same branch in real time via WebSocket.	Must	2
FR-4.2	The Cashier and Kitchen applications shall remain able to exchange new orders and status updates when the branch's internet connection is unavailable, provided the branch LAN is intact.	Must	2
FR-4.3	A client that loses and regains its WebSocket connection shall reconcile its local state against the current active-orders list rather than relying solely on missed events.	Must	2 (hardening)
FR-4.4	The edge server shall buffer state changes that occur while the cloud backend is unreachable and replay them once connectivity is restored, without data loss or duplication.	Must	2 (cloud tier)

Table 3.4 — Real-time sync requirements. FR-4.3 and FR-4.4 extend the built sync engine and are not yet implemented (see Section 6.4).

3.5 Hardware Integration

ID	Requirement	Priority	Phase
FR-5.1	The system shall print a kitchen ticket to a network (TCP port 9100) or USB thermal printer when an order is sent.	Must	3
FR-5.2	The system shall print a customer receipt on payment completion.	Must	3
FR-5.3	The system shall trigger a cash drawer kick command on cash payment.	Should	3
FR-5.4	The system shall integrate with a card payment terminal SDK to initiate, confirm, and record card payments.	Must	3

ID	Requirement	Priority	Phase
FR-5.5	Hardware commands (print, drawer kick, payment) shall be issued from the branch edge server, not directly from the browser client.	Must	3

Table 3.5 — Hardware integration requirements (Phase 3, not yet implemented).

3.6 Inventory & Add-ons

ID	Requirement	Priority	Phase
FR-6.1	The system shall support recipe definitions linking a menu item to the ingredient quantities it consumes.	Must	4
FR-6.2	The system shall automatically deduct ingredient stock when an order item is placed.	Must	4
FR-6.3	The system shall alert branch staff when an ingredient's stock falls below a configurable reorder level.	Should	4
FR-6.4	The system shall support an end-of-day reconciliation report per branch.	Should	4
FR-6.5	The system shall support QR-code-based customer self-ordering for a table.	Could	4

Table 3.6 — Inventory and add-on requirements (Phase 4, not yet implemented).

4. External Interface Requirements

4.1 User Interfaces

- Cashier (POS): touch-optimized screen with table selector, category-filtered menu grid, and a running cart/ticket panel; optimistic UI updates so item taps feel instant.
- Kitchen (KDS): ticket-board layout with one card per active order, grouped line items, per-line status controls, and an elapsed-time indicator per ticket.
- Both interfaces shall clearly indicate connection status to the branch edge server.

4.2 Hardware Interfaces

Device	Interface	Purpose
Network thermal printer	Raw TCP, port 9100	Kitchen ticket and receipt printing
USB thermal printer	WebUSB or local driver bridge	Kitchen ticket and receipt printing
Cash drawer	ESC/POS kick command via printer connection	Opens on cash payment
Card payment terminal	Vendor SDK / local network API (e.g., Stripe Terminal, Adyen)	Card payment capture and confirmation

Table 4.1 — Hardware interfaces (Phase 3).

4.3 Software Interfaces

The branch edge server exposes a REST API for order, menu, and table operations, and a WebSocket endpoint for real-time event broadcast. The full endpoint reference for the implemented (Phase 1–2) API is provided in Appendix A. In the planned cloud tier, the edge server will additionally act as a client of a cloud REST API for replication.

4.4 Communication Interfaces

- Client ↔ Edge server: HTTP (REST) and WebSocket (WSS in production), branch LAN.
- Edge server ↔ Cloud backend (planned): HTTPS (REST/gRPC) for replication, over WAN.

5. Non-Functional Requirements

5.1 Performance

- NFR-1: Order-to-kitchen-ticket latency shall not exceed one second under normal branch LAN conditions.
- NFR-2: The Cashier menu and cart interactions shall respond within 100ms perceived latency (optimistic local UI updates).

5.2 Reliability & Availability

- NFR-3: Cashier-to-Kitchen order flow shall continue functioning during a WAN/internet outage.
- NFR-4: A dropped WebSocket connection shall automatically reconnect without requiring a manual page refresh.
- NFR-5: No order or payment shall be lost or duplicated as a result of a network interruption (see FR-4.4 and the idempotency requirement in Section 5.3).

5.3 Security

- NFR-6: Tenant data shall be isolated such that no query can return another tenant's data, enforced at the database layer in the cloud tier.
- NFR-7: All API and WebSocket access shall require authentication before this system is used with real customer or payment data (not yet implemented in the Phase 1-2 reference build — see Section 6.4).
- NFR-8: Payment operations shall use a client-generated idempotency key so that a retried request cannot result in a duplicate charge.

5.4 Scalability

- NFR-9: The architecture shall support additional branches by deploying additional edge servers without changes to client application code.
- NFR-10: The database schema shall support an arbitrary number of tenants and branches without structural change.

5.5 Usability

- NFR-11: The Cashier interface shall be operable on a touchscreen without a keyboard or mouse for all core ordering actions.
- NFR-12: The Kitchen interface shall be legible from typical kitchen viewing distances (large type, high-contrast status colors).

5.6 Maintainability

- NFR-13: The client application (Cashier and Kitchen) shall share a single codebase, differing only by active view/route.
- NFR-14: The local and cloud database schemas shall remain structurally identical to keep replication a direct row-level operation.

6. System Architecture

6.1 Topology

The system is organized into three tiers: a planned cloud backend, a branch-local edge server, and browser-based clients. The edge server is the real-time source of truth for a branch: it hosts a local WebSocket hub so the Cashier and Kitchen applications exchange orders and status updates directly over the branch LAN, without a round trip to the cloud. Full topology, order-flow, and schema diagrams are provided in the companion System Architecture document; the key structural decision is summarized below.

6.2 Topology Decision & Rationale

The two source planning documents proposed different real-time sync topologies. This specification adopts the Dedicated Branch Edge Server model, for the reasons summarized in Table 6.1.

Criterion	Edge Server (Master Plan)	Browser-direct RxDB (Blueprint)
Offline LAN sync	Reliable — local WebSocket hub does not depend on WAN	Depends on WebRTC P2P, which is unreliable across NAT/guest Wi-Fi
Hardware access	Direct raw TCP/USB access from the edge process	Not possible from a browser for network (port 9100) printers
Installation footprint	One additional local service per branch	Zero install — browser only
Decision	Adopted	Not adopted

Table 6.1 — Topology comparison and rationale.

6.3 Technology Stack

Layer	Technology	Notes
Frontend	React + Vite, Zustand for state	Shared codebase for Cashier and Kitchen views
Edge server	Node.js (Fastify) + ws	REST API and WebSocket hub, branch-local
Branch database	SQLite	Stands in for a branch-local replica; schema matches the cloud model
Cloud backend (planned)	Node.js/Go, PostgreSQL (RLS), Redis	Multi-tenant system of record, cross-branch reporting

Table 6.2 — Technology stack, as implemented.

6.4 Known Gaps Against This Specification

The following requirements are specified above but not yet satisfied by the reference implementation, and should be prioritized before production use:

- FR-4.3 / FR-4.4 — reconnect-time state resync and the cloud replication outbox are not implemented; the edge server is currently the sole source of truth with no cloud tier.
- NFR-7 — there is no authentication on the REST API or WebSocket endpoint.

- NFR-8 — the payments table has an idempotency_key column, but no code path enforces uniqueness yet.
- FR-5.x and FR-6.x (Phases 3 and 4) — not started.

7. Data Requirements

The schema below reflects the multi-tenant data model as implemented in the reference codebase (backend/src/db.js), which mirrors the cloud PostgreSQL design from both source documents. Primary keys are marked PK, foreign keys FK.

TENANTS		
Field	Type	Description
id	string (PK)	Unique tenant identifier
name	string	Tenant / brand name
subscription_tier	string	Billing tier
created_at	timestamp	Record creation time

Table 7.1 — TENANTS.

BRANCHES		
Field	Type	Description
id	string (PK)	Unique branch identifier
tenant_id	string (FK)	Owning tenant
name	string	Branch name
currency	string	Branch currency code
timezone	string	Branch timezone
tax_no	string	Tax registration number

Table 7.2 — BRANCHES.

TABLES		
Field	Type	Description
id	string (PK)	Unique table identifier
branch_id	string (FK)	Owning branch
table_number	string	Table label shown to staff
zone	string	Seating zone (e.g., Indoor, Terrace)
capacity	integer	Seating capacity
status	string	available occupied dirty

Table 7.3 — TABLES.

MENU_ITEMS		
Field	Type	Description

MENU_ITEMS		
id	string (PK)	Unique menu item identifier
branch_id	string (FK)	Owning branch
name	string	Item name
category	string	Menu category
price	real	Unit price
tax_rate	real	Applicable tax rate
is_available	boolean	Whether the item is currently orderable

Table 7.4 — MENU_ITEMS.

ORDERS		
Field	Type	Description
id	string (PK)	Unique order identifier
branch_id	string (FK)	Owning branch
table_id	string (FK)	Associated table (null for takeaway)
order_type	string	dine_in takeaway
status	string	open served paid void
total	real	Order total
version	integer	Optimistic-lock version (reserved for FR-4.x)

Table 7.5 — ORDERS.

ORDER_ITEMS		
Field	Type	Description
id	string (PK)	Unique order-item identifier
order_id	string (FK)	Parent order
item_id	string (FK)	Referenced menu item
item_name	string	Denormalized item name at time of order
quantity	integer	Quantity ordered
modifiers_json	string (JSON)	Item modifiers/customizations
status	string	placed in_prep ready served

Table 7.6 — ORDER_ITEMS.

PAYMENTS		
Field	Type	Description

PAYMENTS		
id	string (PK)	Unique payment identifier
order_id	string (FK)	Associated order
amount	real	Amount charged
method	string	cash card
status	string	Payment status
idempotency_key	string (unique)	Prevents duplicate charge on retry

Table 7.7 — PAYMENTS.

Phase 4 introduces two additional entities not yet present in the reference schema: `inventory_items` (`id`, `branch_id`, `unit`, `stock_qty`, `reorder_level`) and `recipe_items` (`id`, `menu_item_id`, `inventory_item_id`, `qty_used`), linking menu items to the ingredient stock they consume.

Appendix A — REST API Reference (Phase 1-2, Implemented)

All endpoints are served by the branch edge server. Requests and responses use JSON. This reflects the API as implemented in backend/src/server.js.

Method	Endpoint	Purpose
GET	/api/branch	Returns the branch identifier for this edge server instance.
GET	/api/menu?branch_id=	Lists available menu items for a branch.
GET	/api/tables?branch_id=	Lists tables for a branch, including current status.
GET	/api/orders?branch_id=&status=	Lists orders for a branch; status=active filters to open orders.
POST	/api/orders	Creates a new order with one or more line items; broadcasts order.created.
PATCH	/api/orders/:orderId/items/:itemId	Updates a single line item's status; broadcasts order_item.updated.
PATCH	/api/orders/:orderId	Updates order-level status (e.g., paid, void); broadcasts order.updated.
WS	/ws?branch_id=&role=	WebSocket connection for real-time event delivery to POS or KDS clients.

Table A.1 — Implemented REST and WebSocket surface.

A.1 Order Status Lifecycle

Each order line item advances through a fixed sequence of statuses, driven by kitchen staff action on the KDS: Placed → In Prep → Ready → Served. An order's own status remains open until every line item reaches Served, at which point it is automatically set to served and removed from the active board.